Language Specification and Compiler Reference

x86-64 Linux Native Compiler · NASM Backend · Version 1.0 (Draft)

Prepared October 23, 2026 · Source basis: `Generator.h`

Document Control

Field	Value
Title	Language Specification and Compiler Reference
Subject	Code generation back end and language semantics
Target platform	x86-64, Linux (System V syscall ABI), ELF64
Assembler	NASM (Netwide Assembler)
Status	Draft 1.0
Audience	Language users, compiler contributors, reviewers

How to read this document

This document is organised from the general to the specific. Chapters 1 to 3 explain what the language is and how programs become executables. Chapters 4 to 7 describe the language itself: values, variables, expressions and operators. Chapter 8 is the built-in function reference and is the longest chapter. Chapters 9 to 12 cover statements, input and output, arrays and the memory model. Chapters 13 to 15 describe compiler internals, error behaviour and known limitations. Chapter 16 contains worked examples, and the appendices provide quick-reference tables.

Source basis and conventions

Every behavioural statement in this document is derived from the code generator (`Generator.h`). The tokenizer and parser were not part of the material reviewed. Consequently:

- **Semantics** (what each construct computes, which registers it uses, which edge cases it handles) are authoritative.
- **Surface syntax** (exact keyword spellings, punctuation, operator precedence) is not visible from the generator. Code samples in this document use an *illustrative notation* in which each construct is written with a lower-case name that mirrors its syntax-tree node (for example `sum(...)`, `gcd(a, b)`). Replace these with the exact spellings defined in your tokenizer when publishing.
- Passages marked **Note** highlight behaviour that is surprising or that should be verified. Passages marked **Limitation** describe a boundary of the current implementation.

Typographic conventions: `monospace` denotes code, register names, labels and identifiers. *Italic* denotes the first use of a defined term.

---

1. Introduction

1.1 Purpose

This language is a small, statically compiled, integer-oriented programming language. A program written in it is translated by the compiler into NASM assembly for x86-64 Linux, assembled into an object file, and linked into a standalone executable with no runtime library and no dependency on the C standard library. All input, output and process control are performed through direct Linux system calls.

The language is deliberately compact. It combines a conventional core (variables, arithmetic, comparisons, conditionals, counted loops and fixed-size arrays) with an unusually rich set of *built-in expression functions* that implement common numeric routines directly in generated machine code: greatest common divisor, primality testing, integer square root, factorial, Fibonacci numbers, digit manipulation, palindrome testing, hardware random numbers and wall-clock time.

1.2 Design goals

1. **Transparency.** Every construct maps to a short, readable sequence of assembly instructions. A reader can open the generated `.asm` file and follow it.
2. **Self-containment.** The produced binary uses only the Linux kernel interface. There is no heap, no garbage collector and no external library.
3. **Predictability.** Execution cost is determined by the code written. Loops, divisions and syscalls are explicit in the generated output.
4. **Teachability.** The built-ins turn classic exercises (primes, GCD, palindromes, Fibonacci) into single expressions, which makes the language well suited to demonstrating algorithms and compiler construction.

1.3 Non-goals

The language does not currently aim to provide floating-point arithmetic, dynamic memory, user-defined functions, strings as first-class values, or portability beyond x86-64 Linux. Chapter 15 lists the resulting limitations in detail.

1.4 Intended audience

Reader	Most useful chapters
Programmer writing programs in the language	4, 5, 6, 7, 8, 9, 10, 16
Contributor extending the compiler	3, 12, 13, 14, 15, Appendices
Reviewer or maintainer	2, 14, 15

---

2. Language Overview

2.1 At a glance

Characteristic	Description
Paradigm	Imperative, statement-oriented
Typing	Single machine word (signed 64-bit integer) for all runtime values
Compilation	Ahead of time, to NASM assembly
Evaluation	Strict, left to right, stack-based
Scoping	Block scope for `if` and `for` bodies
Aggregates	Fixed-size one-dimensional integer arrays with bounds checking
Control flow	`if` / `else`, counted `for` loop, program exit
Mutability	Scalar variables are declared once; array elements are assignable
I/O	Decimal integer input from standard input; integer and string output to standard output
Runtime	None (no libc, no heap)

2.2 Feature summary

The code generator implements the following families of constructs.

Literals: integer literals, double literals (stored as raw IEEE-754 bit patterns), string literals.

Names: scalar identifiers, array element access, array length.

Operators: addition, subtraction, multiplication, signed division, and the six comparison operators `==`, `!=`, `<`, `>`, `<=`, `>=`, plus a dedicated equality node.

Built-in functions: `sum`, `even`, `abs`, `midval`, `max`, `min`, `fact`, `sqrt`, `gcd`, `prime`, `nth fibonacci`, `random`, `palindrome`, `digit sum`, `count digits`, `power`, `input`, and `time utc`.

Statements: `let` (scalar), `let` (array, with or without initialiser), array element assignment, `if` / `else`, `for`, `print`, `exit`.

2.3 A first look

The following program, written in illustrative notation, reads a number and reports whether it is prime.

```
let n = input
let p = prime(n)
print p
exit 0
```

The compiler produces roughly forty lines of assembly for this program: the `input` routine, the trial-division loop for `prime`, a decimal conversion routine for `print`, and the `exit` syscall.

2.4 Execution model in one paragraph

Execution begins at the label `_start`. Statements run in source order. Every expression evaluates by emitting instructions that leave exactly one 64-bit value on the machine stack. Variables are not stored in registers or a fixed data area: each variable is a slot on the stack, addressed relative to `rsp`. When a block ends, its slots are released by adding to `rsp`. If control reaches the end of the program, the compiler-supplied epilogue calls `exit(0)`.

---

3. Toolchain and Compilation Pipeline

3.1 Pipeline stages

Stage	Component	Input	Output
1	Tokenizer	Source text	Token stream
2	Parser (`Parser.h`)	Tokens	`NodeProgram` syntax tree
3	Generator (`Generator.h`)	`NodeProgram`	NASM assembly text
4	NASM	Assembly text	ELF64 object file
5	Linker (`ld`)	Object file	Executable

The `Generator` class is constructed from a `NodeProgram` and exposes a single public entry point, `generate()`, which returns the complete assembly listing as a string. The generator performs a single pass over the tree; there is no intermediate representation and no optimisation stage.

3.2 Building a program

Assuming the compiler writes its output to `out.asm`:

```
nasm -f elf64 out.asm -o out.o
ld out.o -o out
./out
```

The linker needs no libraries because the program declares only `_start` as a global symbol and performs no external calls.

3.3 Host and target requirements

Requirement	Detail
Operating system	Linux, x86-64
Assembler	NASM supporting `elf64`
Linker	GNU `ld` or compatible

Requirement	Detail
CPU feature	`RDRAND` support, required only by programs that use `random`
Compiler build	C++20 (the generator uses `std::bit_cast` from `<bit>`)

3.4 Structure of the generated file

Every output file has the same overall shape.

1. A `.bss` section reserving `input_buffer` (64 bytes) and `print_buffer` (32 bytes).
2. A `.text` section declaring `_start`.
3. The translated statements of the program, in order.
4. A default epilogue that executes `exit(0)`.
5. The shared error handler `array_oob`, which executes `exit(1)`.

String literals add `.data` fragments at the point of use. The generator switches to `section .data`, defines the string, and immediately switches back to `section .text`, so the fragments never interrupt the instruction stream.

3.5 Syscalls used by generated programs

Number	Name	Used by
0	`read`	`input`
1	`write`	`print`
60	`exit`	`exit` statement, epilogue, `array_oob`
228	`clock_gettime`	`time utc`

---

4. Program Structure

4.1 Programs and statements

A program is an ordered list of statements (`NodeProgram::statements`). There is no `main` function, no import mechanism and no declaration section. Statements execute from top to bottom. The only constructs that alter linear flow are `if` / `else`, `for` and `exit`.

4.2 Statement kinds

Statement	Syntax-tree node	Effect
Exit	`NodeStatementExit`	Terminates the process with an exit code
Print	`NodeStatementPrint`	Writes a string literal or a decimal integer to standard output

Statement	Syntax-tree node	Effect
Scalar declaration	`NodeStatementLet`	Introduces a variable initialised with an expression
Array declaration	`NodeStatementArrayLet`	Introduces a fixed-size array
Array assignment	`NodeStatementArrayAssign`	Stores a value into one element
Conditional	`NodeIf`	Executes one of two blocks
Loop	`NodeFor`	Executes a block for each value in a counted range

4.3 Termination

A program ends in one of three ways:

1. an `exit` statement runs, using the given value as the process exit status;
2. control reaches the end of the statement list, and the epilogue exits with status 0;
3. an array access is out of bounds, and `array_oob` exits with status 1.

Because exit codes are truncated by the kernel to eight bits, `exit 256` is observed by the shell as 0.

4.4 Comments and whitespace

Comment syntax and whitespace rules belong to the tokenizer and are not visible to the generator. Note that the compiler itself places `!` in many of its own assembly comments, which is unrelated to source-language syntax.

---

5. Data Types and Values

5.1 The machine word

All runtime values are 64-bit quantities occupying a single stack slot. The language does not distinguish types at run time. How a value is *interpreted* depends on the construct that consumes it.

Interpretation	Used by
Signed integer, two's complement	Arithmetic, comparisons, most built-ins
Boolean (zero is false, non-zero is true)	`if` conditions; results of comparisons and predicates
Address	String literals (pointer to first byte)
Raw bit pattern	Double literals

5.2 Integer values

Integers range from −9 223 372 036 854 775 808 to 9 223 372 036 854 775 807. Arithmetic wraps silently on overflow, as is normal for the underlying `add`, `sub` and `imul` instructions; the compiler inserts no overflow checks.

An integer literal is emitted verbatim as an immediate operand: `mov rax, <literal>`. The literal text is copied from the token, so the assembler is responsible for range checking.

5.3 Boolean values

There is no separate boolean type. Comparison operators and predicate built-ins (`prime`, `palindrome`, `even`) produce `1` for true and `0` for false. A condition in an `if` statement is true when it is anything other than zero. Consequently the following are equivalent in effect:

```
if prime(n) { ... }
if prime(n) != 0 { ... }
```

5.4 Double literals

A double literal is parsed on the host with `std::stod`, reinterpreted as a 64-bit pattern with `std::bit_cast<uint64_t>`, and emitted as a hexadecimal immediate:

```
mov rax, 0x400921fb54442d18      ; 3.14159265358979...
push rax
```

The value is therefore *stored* correctly but is carried in a general-purpose register, and the language provides no floating-point arithmetic. Applying `+`, `-`, `*`, `/` or any built-in to such a value treats the bit pattern as an integer.

Limitation. Double literals are currently a storage-only feature. Printing one produces the decimal value of its bit pattern, not its numeric value.

5.5 String literals

A string literal is placed in the data section and its address is pushed:

```
section .data
str_0 db `Hello`, 0
section .text
    lea rax, [rel str_0]
    push rax
```

The text is wrapped in NASM *backtick* quotation, so C-style escape sequences such as `\n` and `\t` are interpreted by the assembler. Each literal receives a unique label `str_<id>`, with `id` drawn from a monotonically increasing counter. Strings are zero-terminated.

Note. The `print` statement writes exactly `value.size()` bytes, where `value` is the *source* text of the literal. If the literal contains an escape sequence, the assembled string is shorter than the source text, because `\n` is two source characters but one byte. See limitation L-07.

5.6 Arrays

Arrays are fixed-length sequences of 64-bit integers, stored contiguously on the stack. Their length is known at compile time and is recorded in the generator. See Chapter 11.

6. Variables and Scope

6.1 Declaring a variable

A scalar variable is introduced with a `let` statement. The initialiser is any expression.

```
let total = 10 + 5
let root = sqrt(total * total)
```

The generator records the variable name together with the index of the stack slot that will hold its value, then evaluates the initialiser. Because every expression leaves exactly one value on the stack, that value *becomes* the variable; no copy is required.

6.2 Redeclaration

Declaring a name that is already visible, whether as a scalar, an array or a loop variable, is a compile-time error:

```
Error: identifier already used: total
```

The compiler prints this message to standard error and exits with `EXIT_FAILURE`. There is no shadowing: a block cannot reuse a name from an enclosing scope.

6.3 Immutability of scalars

The generator contains no statement for assigning to a scalar after its declaration. Scalars are therefore effectively *single-assignment* values. Mutable state is available through array elements, which can be updated with an array assignment statement, and through the implicit loop counter of `for`, which is managed by the compiler.

6.4 Block scope

The bodies of `if`, `else` and `for` are blocks. Any variable or array declared inside a block is visible only until the end of that block. On leaving a block, the generator:

1. emits `add rsp, N*8` to release the block's stack slots (only if any were created);
2. restores its stack-size counter;
3. restores its variable and array tables to the snapshot taken on entry.

Variables declared *before* a block remain visible *inside* it, so a block can read the enclosing scope's data.

6.5 Use before declaration

Referencing a name that has not been declared yields:

```
Error: undeclared identifier: x
```

and for array operations:

```
Error: undeclared array: a
```

Both are reported at compile time, so a successfully compiled program cannot fail name resolution at run time.

6.6 Lifetime

A variable lives from its declaration until the end of its enclosing block (or the end of the program). Storage is the stack, so there is no cost to allocation beyond the `push` that creates the value.

---

7. Expressions and Operators

7.1 The stack discipline

The central rule of the code generator is:

> *Generating an expression emits instructions that, when executed, leave exactly one new 64-bit value on top of the stack and change nothing else that is observable to the program.*

A composite expression is therefore compiled by generating each operand in turn (each pushes one value), popping them into registers, performing the operation, and pushing the single result.

For `a - b`, the emitted code has this shape:

```
    ...code for a...        ; pushes a
    ...code for b...        ; pushes b
    pop rbx                 ; b
    pop rax                 ; a
    sub rax, rbx
    push rax
```

The left operand is always generated before the right operand, so evaluation order is left to right.

7.2 Binary operators

Operator	Meaning	Instruction sequence	Result
`+`	Addition	`add rax, rbx`	Sum, wraps on overflow
`-`	Subtraction	`sub rax, rbx`	Difference
`*`	Multiplication	`imul rax, rbx`	Low 64 bits of product
`/`	Signed division	`cqo` · `idiv rbx`	Quotient, truncated toward zero
`==`	Equal	`cmp` · `sete al` · `movzx rax, al`	1 or 0
`!=`	Not equal	`cmp` · `setne al` · `movzx`	1 or 0
`<`	Less than (signed)	`cmp` · `setl al` · `movzx`	1 or 0
`>`	Greater than (signed)	`cmp` · `setg al` · `movzx`	1 or 0

Operator	Meaning	Instruction sequence	Result
`<=`	Less or equal (signed)	`cmp` · `setle al` · `movzx`	1 or 0
`>=`	Greater or equal (signed)	`cmp` · `setge al` · `movzx`	1 or 0

All comparisons are *signed*. The language has no unsigned comparison operators.

7.3 Division semantics

Division uses `cqo` to sign-extend `rax` into `rdx:rax`, then `idiv`. The quotient truncates toward zero, so `-7 / 2` is `-3`. The language has no remainder operator in the generator, although the remainder is produced in `rdx`.

Limitation. The generator does not test for a zero divisor. Dividing by zero, or dividing `INT64_MIN` by `-1`, raises a hardware divide exception and the process is terminated by `SIGFPE`.

7.4 The dedicated equality node

In addition to `==` inside the general binary node, the tree has a separate `NodeEqual` with the same behaviour: compare, `sete al`, `movzx`. The two are interchangeable at run time. The duplication is historical and harmless.

7.5 Precedence and associativity

Precedence and associativity are determined by the parser, which was not reviewed. The generator itself imposes no precedence: it simply walks whatever tree it is given. Use parentheses wherever the intended grouping is not obvious.

7.6 Variable references

Reading a scalar is a single instruction that copies the slot to the top of the stack:

```
    push QWORD [rsp + offset]
```

The offset is computed as `(stack_size − slot − 1) × 8`, using the number of slots currently on the stack. Because that count changes while an expression is partially evaluated, the offset is recalculated at every use. This is what allows `a + b` to work correctly even though evaluating `a` has already pushed one slot before `b` is read.

Note. Using an *array name* in a position where a scalar is expected is not rejected. Because arrays are registered in the same variable table, the name resolves to the slot of element 0 and yields that element's value.

7.7 Array element access

`name[index]` generates the index expression, pops it into `rcx`, calls the shared address routine (which performs a bounds check and leaves the element address in `rbx`), loads the value and pushes it. The detailed mechanism is given in Chapter 11.

7.8 Array length

The length of an array is a compile-time constant, so the `len` construct reduces to `mov rax, <length>` followed by a push. It costs one instruction and cannot fail at run time.

7.9 Expression cost model

Construct	Approximate run-time cost
Literal, variable, `len`	O(1), one or two instructions
`+ - *`, comparisons, `even`, `abs`, `min`, `max`	O(1)
`/`, `midval`	O(1), one `idiv` (tens of cycles)
`sum` of k arguments	O(k)
`digit sum`, `count digits`, `palindrome`	O(d), d is the number of decimal digits
`fact`, `power`, `nth fibonacci`	O(n), linear in the argument
`sqrt`	O(log n), binary search
`gcd`	O(log min(a, b))
`prime`	O(√n), trial division
`random`	Expected O(1), loops until RDRAND succeeds and the sample is accepted
`input`	One `read` syscall plus O(d) parsing
`time utc`	One `clock_gettime` syscall

---

8. Built-in Function Reference

The built-in functions are the distinguishing feature of the language. Each one is an *expression*: it can appear anywhere an expression is allowed, it may be nested inside other expressions, and it always leaves exactly one value on the stack. Every built-in evaluates its argument expressions first (left to right), pops them into registers, performs its computation with inline code, and pushes the result.

8.1 Summary table

Built-in	Syntax-tree node	Arguments	Result	Complexity
`sum`	`NodeSum`	1 or more expressions	Sum of all arguments	O(k)
`even`	`NodeEven`	1	1 if even, else 0	O(1)
`abs`	`NodeAbs`	1	Absolute value	O(1)

Built-in	Syntax-tree node	Arguments	Result	Complexity
`midval`	`NodeMidVal`	1	Value divided by 2 (truncating)	O(1)
`max`	`NodeMax`	2	Larger of two values	O(1)
`min`	`NodeMin`	2	Smaller of two values	O(1)
`fact`	`NodeFact`	1	Factorial	O(n)
`sqrt`	`NodeSqrt`	1	Integer (floor) square root	O(log n)
`gcd`	`NodeGCD`	2	Greatest common divisor	O(log n)
`prime`	`NodePrime`	1	1 if prime, else 0	O(√n)
`nth fibonacci`	`NodeNthFibonacci`	1	n-th Fibonacci number	O(n)
`random`	`NodeRandom`	2	Random integer in a closed range	Expected O(1)
`palindrome`	`NodePalindrome`	1	1 if decimal palindrome, else 0	O(d)
`digit sum`	`NodeDigitSum`	1	Sum of decimal digits	O(d)
`count digits`	`NodeCountDigits`	1	Number of decimal digits	O(d)
`power`	`NodePower`	2 (base, exponent)	base raised to exponent	O(e)
`input`	`NodeInput`	none	Integer read from standard input	O(d)
`time utc`	`NodeTimeUTC`	none	Seconds since the Unix epoch	O(1)

8.2 Common behaviour

The following rules apply to every built-in unless a section says otherwise.

1. **One result.** Each built-in pushes exactly one 64-bit value.
2. **Register scratch space.** Built-ins freely overwrite `rax`, `rbx`, `rcx`, `rdx` and, where noted, `r8` to `r12`. No value lives in a register across statements or between sub-expressions, because every operand is parked on the stack. This is why built-ins can be nested without conflicts.
3. **Unique labels.** Built-ins that contain branches obtain a fresh numeric identifier from a per-built-in counter, so a program may use the same built-in any number of times. The two exceptions are described in limitations L-01 and L-02.

4. **Integer semantics.** Arguments and results are signed 64-bit integers. Overflow wraps silently.
5. **Nesting.** Arguments may themselves be built-in calls: `gcd(fact(5), power(2, 8))` is valid.

8.3 `sum`

Purpose. Adds any number of values.

Signature. `sum(e1, e2, ..., ek)` with `k ≥ 1`.

Semantics. The first argument is generated and left on the stack. For every further argument, the generator evaluates it, pops the pair into `rbx` and `rax`, adds, and pushes the running total. The result is the arithmetic sum, wrapped to 64 bits.

Edge cases.

- A single argument returns that argument unchanged.
- The generator indexes `args[0]` without checking that the list is non-empty. The parser must guarantee at least one argument.

Generated code (per additional argument).

```
    pop rbx
    pop rax
    add rax, rbx
    push rax
```

Example.

```
let total = sum(4, 8, 15, 16, 23, 42)
print total          ; 108
```

8.4 `even`

Purpose. Tests the parity of an integer.

Semantics. Returns `1` if the argument is even and `0` if it is odd. The implementation isolates the lowest bit with `and rax, 1` and inverts it with `xor rax, 1`.

Edge cases. Because two's complement preserves parity, negative numbers behave correctly: `even(-4)` is `1` and `even(-3)` is `0`. Zero is even.

Complexity. Two instructions, no branches, no labels.

Example.

```
let n = 10
if even(n) {
    print "even"
} else {
    print "odd"
}
```

8.5 `abs`

Purpose. Computes the absolute value.

Semantics. Compares the argument with zero; if it is negative, negates it. A fresh label `abs_done_<id>` marks the join point.

```
    cmp rax, 0
    jge abs_done_N
    neg rax
abs_done_N:
```

Edge cases. `abs(INT64_MIN)` is `INT64_MIN`, because the positive counterpart is not representable in 64 bits. All other inputs behave as expected.

8.6 `midval`

Purpose. Halves a value.

Semantics. Loads `2` into `rbx`, sign-extends with `cqo`, and performs `idiv rbx`. The quotient is pushed. The result is therefore the argument divided by two with truncation toward zero: `midval(7)` is `3`, `midval(-7)` is `-3`, `midval(1)` is `0`.

Note. The name suggests a midpoint, but the construct operates on a single operand. To compute the midpoint of two values use `midval(a + b)`, remembering that the sum can overflow for very large inputs.

8.7 `max` and `min`

Purpose. Select the larger or smaller of two values.

Semantics. Both operands are generated, popped into `rax` (left) and `rbx` (right), and compared as signed integers.

Built-in	Branch	Behaviour
`max`	`jge max_done_<id>`	Keeps `rax` if `rax ≥ rbx`, otherwise replaces it with `rbx`
`min`	`jle min_done_<id>`	Keeps `rax` if `rax ≤ rbx`, otherwise replaces it with `rbx`

Edge cases. Comparison is signed, so `max(-1, 0)` is `0`. When the operands are equal the result is that value. Both operands are always evaluated, even if one would not be needed.

Example.

```
let clamped = min(max(x, 0), 100)     ; clamp x into [0, 100]
```

8.8 `fact`

Purpose. Computes the factorial of a non-negative integer.

Semantics. The argument is copied to `rcx` and the accumulator `rax` starts at `1`. If `rcx ≤ 1` the result is `1`. Otherwise the loop multiplies `rax` by `rcx` and decrements `rcx` until `rcx` is no longer greater than `1`.

```
    mov rcx, rax
    mov rax, 1
    cmp rcx, 1
    jle fact_done_N
fact_loop_N:
    imul rax, rcx
    dec rcx
    cmp rcx, 1
    jg fact_loop_N
fact_done_N:
```

Edge cases.

- `fact(0)` and `fact(1)` are `1`.
- Negative arguments also return `1`, because the first comparison exits immediately.
- The largest argument whose factorial fits in a signed 64-bit integer is `20` (`20! = 2 432 902 008 176 640 000`). `fact(21)` and above wrap silently and yield meaningless values.

n	fact(n)
0	1
5	120
10	3 628 800
20	2 432 902 008 176 640 000
21	overflow (wraps)

8.9 `sqrt`

Purpose. Computes the integer square root, that is the largest integer `r` such that `r × r ≤ n`.

Algorithm. Binary search. The invariant is `low² ≤ n < high²`, with `low = 0` and `high = n + 1` initially. Each iteration forms `mid = (low + high) >> 1`, squares it, and moves either bound to `mid`. The loop stops when `high − low ≤ 1`, at which point `low` is the answer.

Register	Role
`r8`	Original number
`rbx`	`low`
`rcx`	`high`
`rdx`	`mid`
`rsi`	`mid × mid`

Edge cases.

- Arguments less than or equal to zero yield `0` through the label `sqrt_zero_<id>`.
- `sqrt(1)` is `1`, `sqrt(15)` is `3`, `sqrt(16)` is `4`.
- The product `mid × mid` is computed with a signed 64-bit multiply. For very large arguments (roughly above the range of 32-bit integers) the first midpoint is large enough for the product to overflow, which can steer the search the wrong way. Results are reliable for arguments up to about four billion. See limitation L-06.

Complexity. About 64 iterations at most, each a handful of instructions.

8.10 `gcd`

Purpose. Computes the greatest common divisor of two integers.

Algorithm. The classical Euclidean algorithm. With `rax = a` and `rbx = b`, each iteration replaces the pair `(a, b)` by `(b, a mod b)` until `b` is zero. The implementation saves `b` in `rcx`, performs `cqo` and `idiv rbx` (which leaves `a mod b` in `rdx`), then sets `rax = rcx` and `rbx = rdx`.

Edge cases.

Call	Result
`gcd(12, 18)`	6
`gcd(7, 0)`	7
`gcd(0, 7)`	7
`gcd(0, 0)`	0
`gcd(-12, 18)`	May be negative; see below

`idiv` produces a remainder with the sign of the dividend, so when an input is negative the final value in `rax` can be negative. Programs that need a non-negative GCD should wrap the arguments with `abs`: `gcd(abs(a), abs(b))`.

Complexity. O(log min(|a|, |b|)) iterations.

8.11 `prime`

Purpose. Primality test.

Algorithm. Trial division. Numbers below 2 are rejected immediately. Otherwise the divisor `rcx` starts at 2. On each iteration:

1. if `rcx × rcx > n`, the number is prime;
2. otherwise divide `n` by `rcx`; if the remainder is zero, the number is composite;
3. increment `rcx` and repeat.

Result. `1` for prime, `0` for not prime.

Examples.

Call	Result
`prime(-5)`, `prime(0)`, `prime(1)`	0
`prime(2)`, `prime(3)`	1
`prime(4)`	0
`prime(97)`	1

Performance. Every integer from 2 up to √n is tried, including even numbers. For a 10-digit prime this is about 100 000 divisions, which is instantaneous; for numbers near 10^{18} it is about a billion divisions and takes several seconds. Skipping even divisors would halve the work and is listed among the suggested improvements in Appendix E.

8.12 `nth fibonacci`

Purpose. Returns the n-th Fibonacci number with the conventional indexing `F(0) = 0`, `F(1) = 1`.

Algorithm. Iterative. The pair `(r8, r9)` holds `(F(k), F(k+1))`; each iteration computes the next term in `r10` and shifts the pair. The loop counter is `rcx`.

Edge cases.

- Negative arguments return `0`.
- Arguments `0` and `1` return themselves.
- The largest index that fits in a signed 64-bit integer is `92` (`F(92) = 7 540 113 804 746 346 429`). Larger indices wrap.

n	F(n)
0	0
1	1
2	1
10	55
20	6 765
50	12 586 269 025
92	7 540 113 804 746 346 429

8.13 `random`

Purpose. Produces a uniformly distributed integer in the closed interval `[lower, upper]`.

Signature. `random(lower, upper)`.

Entropy source. The x86 `RDRAND` instruction. The generated code retries (`jnc random_retry_<id>`) until the processor reports success through the carry flag.

Algorithm. The range size is `upper - lower + 1`. The sample from `RDRAND` is accepted only if it falls at or below a threshold, to avoid modulo bias (the technique known as *rejection sampling*). The accepted value is reduced modulo the range size and offset by `lower`.

Edge cases.

- If `lower > upper` the result is `0`.
- If `lower == upper` the result is `lower`.
- The program requires a CPU that implements `RDRAND`. On a CPU without it the instruction raises an illegal-instruction fault.
- The full 64-bit range makes the range size wrap to zero and causes a division fault.

Note. The threshold computation performs its division on `rax` (which still holds the lower bound at that point) rather than on the constant `2^64`. The comments in the generator describe the intended computation. As a result the bias correction is not exact, and for `lower = 0` it is effectively disabled. The output is still always inside the requested range. See limitation L-08.

8.14 `palindrome`

Purpose. Tests whether the decimal representation of a number reads the same forwards and backwards.

Algorithm. The digits of the argument are peeled off with repeated division by 10 and accumulated in reverse (`reversed = reversed × 10 + digit`). The original value, kept in `r8`, is then compared with the reversed value in `r9`.

Edge cases.

- Negative numbers are never palindromes (result `0`).
- `0` is a palindrome, as is every single-digit number.
- Numbers whose reversal does not fit in 64 bits (about 19 digits and above) can produce false results through overflow.

Call	Result
`palindrome(121)`	1
`palindrome(1221)`	1
`palindrome(123)`	0
`palindrome(10)`	0
`palindrome(-121)`	0

8.15 `digit sum` and `count digits`

These two built-ins both walk the decimal digits of the *magnitude* of their argument.

`digit sum` adds the digits. A negative argument is negated first, so `digit sum(-123)` is `6`. The special value `INT64_MIN` is handled correctly because the negated bit pattern is interpreted by the unsigned `div` instruction as 2^{63}.

`count digits` counts them. It uses a do-while loop, so `count digits(0)` is `1`. `count digits(-5)` is `1`, `count digits(1000)` is `4`.

Call	Result
`digit sum(0)`	0
`digit sum(9875)`	29
`digit sum(-123)`	6
`count digits(0)`	1
`count digits(12345)`	5
`count digits(-99)`	2

Note. `count digits` emits the fixed labels `count_digits_loop` without a unique suffix. Using it twice in one program produces a duplicate label error in NASM. See limitation L-01.

8.16 `power`

Purpose. Raises a base to a non-negative integer exponent.

Syntax. `power(base, exponent)`.

Algorithm. Repeated multiplication. The accumulator starts at `1`; the loop multiplies it by the base `exponent` times.

Edge cases.

- `power(x, 0)` is `1` for any `x`, including `0`.
- Results that do not fit in 64 bits wrap silently. `power(2, 62)` is correct; `power(2, 64)` is `0`.
- A **negative exponent** is not rejected. The loop counter is decremented from a negative value and never reaches zero, so the program effectively does not terminate. See limitation L-04.

Complexity. O(exponent). Exponentiation by squaring would reduce this to O(log exponent).

8.17 `input`

Purpose. Reads one integer from standard input.

Behaviour.

1. Executes `read(0, input_buffer, 64)`.
2. If the call returns zero or an error, the result is `0`.
3. If the first byte is `-`, the sign is remembered and parsing starts at the next byte.
4. Decimal digits are accumulated (`result = result × 10 + digit`). Parsing stops at the first newline, carriage return, or non-digit.
5. The result is multiplied by the sign and pushed.

Registers used. `r8` (buffer), `r9` (bytes read), `r10` (index), `r11` (accumulator), `r12` (sign), `rcx` (current character).

Notes.

- Leading whitespace or a leading `+` stops parsing at once and yields `0`.
- Each `input` expression performs *one* `read` syscall of up to 64 bytes. When standard input is a terminal, the kernel delivers one line per read, which is what interactive programs expect. When input is piped from a file that contains several lines, a single read can consume several of them, and the remainder is discarded. See limitation L-09.
- Values with more than 19 digits overflow.

8.18 `time utc`

Purpose. Returns the current time as whole seconds since 1970-01-01 00:00:00 UTC.

Mechanism. Reserves 16 bytes of stack for a `timespec`, then calls `clock_gettime(CLOCK_REALTIME, &ts)` (syscall 228, clock identifier 0 passed in `rdi`). On success, it loads the seconds field and releases the 16 bytes. On failure it releases the space and returns `0`.

Note. This built-in uses the fixed labels `time_utc_error` and `time_utc_done`, so it can appear only once in a program. See limitation L-02. The `sub rsp, 16` and `add rsp, 16` pair is balanced within the construct, so the generator's own stack bookkeeping is unaffected.

8.19 Composing built-ins

Because every built-in is an ordinary expression, built-ins compose freely. Some useful idioms follow.

Goal	Expression
Number of digits in a factorial	`count digits(fact(10))`
Is a number a perfect square?	`sqrt(n) * sqrt(n) == n`

Goal	Expression
Least common multiple	`a / gcd(a, b) * b`
Clamp to a range	`min(max(x, lo), hi)`
Sum of the first n squares	use a `for` loop, see Chapter 16
Are two numbers coprime?	`gcd(a, b) == 1`
Random die roll	`random(1, 6)`
Is a prime also a palindrome?	`prime(n) * palindrome(n)` (product is 1 only when both are 1)

The last idiom illustrates a general technique: since predicates produce `0` or `1`, multiplication acts as logical AND and `max` acts as logical OR.

---

9. Statements

This chapter describes each statement in the order they appear in the code generator. A statement never leaves a value for the program to use; the net effect on the stack is either nothing (for example `print`) or the creation of a variable slot (for example `let`).

9.1 `exit`

Purpose. Terminates the process immediately with a chosen status code.

Form. `exit <expression>`

Generated code.

```
    ...code for the expression...   ; pushes the value
    mov rax, 60                     ; syscall number for exit
    pop rdi                         ; exit status
    syscall
```

Semantics. The expression is evaluated, and its low eight bits become the exit status observed by the parent process. By convention `0` means success. No code after an `exit` statement in the same control path is executed.

Example.

```
let code = 3 + 4
exit code          ; process status is 7
```

Notes. Every program also receives a default `exit(0)` after the last statement, so an explicit `exit` is needed only for non-zero codes or early termination.

9.2 `print`

Purpose. Writes a value to standard output.

Form. `print <expression>`

The generator distinguishes two cases by inspecting the *syntactic form* of the expression.

Case 1: the expression is a string literal. The literal is generated (placing it in the data section and pushing its address), the address is popped into `rsi`, and a `write(1, rsi, length)` syscall is issued with `length` equal to the number of characters in the source literal. No newline is appended; the programmer includes one in the string where needed.

```
    mov rax, 1          ; write
    mov rdi, 1          ; file descriptor 1 (stdout)
    mov rdx, <length>
    syscall
```

Case 2: any other expression. The expression is evaluated, popped into `rax`, and converted to decimal text by repeated division by 10. The digits are written backwards into `print_buffer`, a newline character is stored at offset 31, and a single `write` syscall emits the digits followed by `\n`.

Consequences.

- Printing a string literal does **not** end the line; printing a number **does**.
- A string stored in a variable cannot be printed as text. `let s = "hi"` followed by `print s` goes down the integer path and prints the string's *address* as a number.
- Only the literal form `print "text"` produces text output.

Limitation. The integer path has three properties that should be understood. First, the conversion uses unsigned division, so a negative number is printed as the unsigned interpretation of its bit pattern (see L-05). Second, the routine uses the local label `.print_loop`, which NASM scopes to the preceding non-local label; two `print` statements with no non-local label between them cause a redefinition error (L-03). Third, the length passed to `write` is computed as `32 - rcx`, which is larger than the digit count plus one (L-05).

Example.

```
print "The answer is:\n"
print 6 * 7
```

9.3 Scalar declaration (`let`)

Purpose. Introduces a new scalar variable.

Form. `let <name> = <expression>`

Compile-time actions.

1. Reject the declaration if `<name>` is already in the variable table (error: *identifier already used*).
2. Record `m_vars[name] = m_stack_size`, which is the index of the slot that the next push will occupy.
3. Generate the expression. Its single pushed value *is* the variable's storage.

Run-time effect. Exactly one `push` is executed (by the final instruction of the expression) and the stack grows by one slot.

Scope. The variable lives until the end of the enclosing block or program.

Note. Because the name is registered *before* the initialiser is generated, an initialiser that mentions the variable being declared (`let x = x + 1`) is not diagnosed as undeclared. It resolves to a slot that does not yet exist and reads unrelated stack contents. Programs should never refer to a variable in its own initialiser.

9.4 Array declaration

Purpose. Introduces a fixed-size array of 64-bit integers.

Forms.

Form	Meaning
`let a = [e0, e1, ..., en-1]`	Array of *n* elements, initialised from the expressions in order
`let a[n]` (sized, uninitialised form)	Array of *n* elements, all set to `0`

The exact spelling of the two forms is parser-defined; the generator distinguishes them by the `has_init` flag on `NodeStatementArrayLet`.

Compile-time actions.

1. Reject the declaration if the name is already in use.
2. Record the slot index `base` of element 0.
3. With an initialiser: generate each element expression in source order. Each leaves exactly one slot, so after *n* expressions the array occupies *n* consecutive slots.
4. Without an initialiser: emit `push 0` *n* times.
5. Register the name in both the variable table (with `base`) and the array table (with the element count).

Notes.

- The array is registered only after its elements have been generated, so an initialiser cannot refer to the array being declared.
- The length is a compile-time constant: an array cannot be resized.
- Element expressions may use variables, built-ins and earlier arrays: `let squares = [a*a, b*b, c*c]`.

9.5 Array element assignment

Purpose. Stores a value into an existing array element.

Form. `<name>[<index>] = <value>`

Evaluation order. The *value* is evaluated first, then the *index*. This is the reverse of the textual order in the statement and matters only when the two expressions have side effects (`input` or `random`).

Generated code.

```
    ...value...              ; push
    ...index...              ; push
    pop rcx                  ; index
    pop rax                  ; value
    (bounds check and address computation, see Chapter 11)
    mov [rbx], rax
```

An index outside `[0, length)` jumps to the shared handler `array_oob`, which terminates the program with exit status 1.

9.6 Conditional (`if` / `else`)

Purpose. Executes one of two blocks depending on a condition.

Form. `if <condition> { ... } else { ... }`, where the `else` block may be empty.

Generated structure.

```
    ...condition...          ; push
    pop rax
    cmp rax, 0
    je if_else_N
    ...then body...
    add rsp, 8*k             ; release then-body variables (if any)
    jmp if_end_N
if_else_N:
    ...else body...
    add rsp, 8*k             ; release else-body variables (if any)
if_end_N:
```

Truth value. Zero is false; every other value is true. Comparison operators and predicate built-ins therefore work directly as conditions.

Scope handling. Before generating either body, the generator takes snapshots of the stack size and of the variable and array tables. After each body, it restores the snapshots and, if the body created variables, emits `add rsp, ...` to pop them. Variables declared inside a branch are therefore invisible after the statement, and the two branches cannot see each other's variables.

Nesting. Conditionals can be nested to any depth. Each receives a unique identifier from `m_if_count`, so labels never collide.

There is no `else if` construct in the generator. An else-if chain is written by placing a conditional inside the else block.

Example.

```
let n = input
if n < 0 {
    print "negative\n"
} else {
    if n == 0 {
        print "zero\n"
    } else {
        print "positive\n"
    }
}
```

9.7 Counted loop (`for`)

Purpose. Repeats a block for each integer in a half-open range.

Form. `for <name> = <start> to <end> { ... }` (illustrative).

Semantics.

- `name` takes the values `start, start+1, ..., end-1`. The *end value is exclusive*.
- If `start >= end` the body does not run.
- The end expression is evaluated **once**, before the loop, and stored on the stack. Changes to variables used in the end expression have no effect on the iteration count.
- The loop variable advances by exactly one per iteration and is incremented in place (`inc QWORD [rsp + offset]`).
- The loop variable name must not already be in use.

Stack layout during the loop.

Slot	Contents
base	Loop variable
base + 1	Evaluated end value
base + 2 …	Variables declared inside the body (released on every iteration)

Generated structure.

```
    ...start...               ; slot: loop variable
    ...end...                 ; slot: end value
for_start_N:
    mov rax, QWORD [rsp + var_offset]
    cmp rax, QWORD [rsp + end_offset]
    jge for_end_N
    ...body...
    add rsp, 8*k              ; release body variables, if any
    inc QWORD [rsp + var_offset]
    jmp for_start_N
for_end_N:
    add rsp, 16               ; release loop variable and end value
```

Body variable cleanup. Variables and arrays declared inside the body are released at the end of *each* iteration, so a loop that declares a variable does not grow the stack as it runs.

Scalar mutability inside loops. Since there is no scalar assignment statement, accumulation across iterations is done through an array element. For example, a running total is kept in a one-element array: `total[0] = total[0] + i`.

Example.

```
let total[1]
for i = 1 to 11 {
    total[0] = total[0] + i
}
print total[0]       ; 55
```

Limitations. Loops count upward by one only. There is no `break`, `continue` or `while` in the generator. A countdown is written by indexing with an expression such as `n - 1 - i`.

---

10. Input and Output

10.1 Overview

Channel	Mechanism	Direction	Data
Standard input	`input` expression	In	One signed decimal integer per call
Standard output	`print` statement	Out	String literals; decimal integers followed by a newline
Process status	`exit` statement	Out	Eight-bit exit code
Standard error	Compiler only	Out	Compile-time diagnostics

Generated programs never write to standard error.

10.2 Reading numbers

The `input` expression is described fully in section 8.17. Three practical points deserve emphasis.

1. **Interactive use works as expected.** When connected to a terminal, each `input` consumes one line.
2. **Invalid text yields zero.** An empty line, end of file, or text that does not begin with a digit or a minus sign produces `0`, with no error indication.
3. **Piped input needs care.** Because `input` reads up to 64 bytes in a single call, supplying several numbers through a pipe or file redirect can lose data; see L-09.

10.3 Writing numbers and text

For presentation, a program usually alternates `print "label: "` with a numeric `print`. Because the numeric `print` always adds a newline, the label and the number appear on *separate lines*:

```
print "Sum:\n"
print sum(a, b, c)
```

There is no construct for printing a number without a trailing newline, nor for printing a string followed by a number on the same line.

10.4 Escape sequences in strings

Because string literals are emitted inside NASM backtick quotes, the escape sequences understood by NASM are available: `\n` (newline), `\t` (tab), `\\` (backslash), `\"` (double quote), `\r` (carriage return) and numeric forms such as `\x41`. A literal backtick in the text would end the quotation early and must not be used.

The length passed to `write` is the length of the *source* text. A string such as `"Hi\n"` has four source characters but assembles to three bytes, so the write would emit one byte beyond the terminator. Section 15 (L-07) covers this and the recommended fix of computing the length from the assembled string.

---

11. Arrays in Depth

11.1 Declaration and storage

An array of *n* elements occupies *n* consecutive stack slots. Element 0 is pushed first and therefore resides at the **highest** address; later elements reside at progressively **lower** addresses. This is the opposite of the convention in most languages, where element 0 has the lowest address, but it follows directly from the way the stack grows downward.

```
let a = [10, 20, 30]

  higher addresses
  +----------------+
  | a[0] = 10      |   <- slot base          (pushed first)
  +----------------+
  | a[1] = 20      |   <- slot base + 1
  +----------------+
  | a[2] = 30      |   <- slot base + 2      (pushed last)   <- rsp
  +----------------+
  lower addresses
```

11.2 Address computation

The shared routine `gen_array_addr` takes the index in `rcx` and leaves the element address in `rbx`. It performs three steps.

1. **Bounds check.** `cmp rcx, 0` / `jl array_oob`, then `cmp rcx, <length>` / `jge array_oob`. The length is an immediate constant taken from the compile-time array table.
2. **Address of element 0.** `lea rbx, [rsp + offset]`, where `offset = (m_stack_size - base - 1) × 8`. This accounts for any temporaries currently above the array.
3. **Walk down.** `shl rcx, 3` (multiply the index by 8) and `sub rbx, rcx`.

Because the generator pops the index into `rcx` *before* calling `gen_array_addr`, the value of `m_stack_size` used in step 2 matches the real stack pointer at that instant. Computing the address before the pop would be off by one slot; the comment in the source ("pop BEFORE computing offsets") records this.

11.3 Reading and writing

Operation	Final instruction(s)
Read `a[i]`	`mov rax, [rbx]` then `push rax`

Operation	Final instruction(s)
Write `a[i] = v`	`mov [rbx], rax`

11.4 Bounds checking

Every access is bounds checked at run time. A violation transfers control to `array_oob`, which executes `exit(1)` without printing a message. A test script can therefore detect an out-of-bounds access by observing exit status 1, but cannot distinguish it from a deliberate `exit 1`.

The check treats the index as a signed 64-bit value, so negative indices are caught by the first comparison. The cost is two compare-and-branch pairs per access.

11.5 The `len` expression

`len(a)` is replaced at compile time by the constant element count. It can therefore be used as a loop bound:

```
let data = [4, 8, 15, 16, 23, 42]
let total[1]
for i = 0 to len(data) {
    total[0] = total[0] + data[i]
}
print total[0]      ; 108
```

11.6 Capacity and lifetime

- Arrays live on the machine stack, whose default limit on Linux is typically eight megabytes. An array of more than about one million elements risks a stack overflow and a segmentation fault.
- An array declared inside an `if` or `for` body is released when the body ends.
- Arrays cannot be copied, compared, returned or resized; only individual elements can be read and written.
- Arrays are one-dimensional. A two-dimensional table is simulated with index arithmetic: `grid[row * width + col]`.

11.7 Worked example: reversing an array in place

```
let a = [1, 2, 3, 4, 5]
let n = len(a)
let tmp[1]
for i = 0 to n / 2 {
    tmp[0] = a[i]
    a[i] = a[n - 1 - i]
    a[n - 1 - i] = tmp[0]
}
for j = 0 to n {
    print a[j]
}
```

The output is `5 4 3 2 1`, one value per line. Note the use of a one-element array as a mutable temporary, a standard idiom in this language.

12. Memory Model and Stack Layout

12.1 Regions of memory

Region	Contents	Created by
`.text`	Program code	Generator output
`.data`	String literals (`str_<id>`)	String literal expressions
`.bss`	`input_buffer` (64 bytes), `print_buffer` (32 bytes)	Generator prologue
Stack	Variables, arrays, loop state, expression temporaries	`push`, `pop`, `add rsp`

There is no heap and no dynamic allocation.

12.2 The stack-size counter

The generator maintains `m_stack_size`, the number of 8-byte slots it believes are currently on the machine stack. `push` increments it and `pop` decrements it, in step with the corresponding emitted instruction. Variables are identified not by an address but by a slot *index* recorded at declaration time. At each use, the generator converts the index to a byte offset from the current `rsp`:

```
offset = (m_stack_size - slot_index - 1) * 8
```

Because the offset is recomputed every time, the same variable has different offsets at different points in an expression, and the code remains correct.

12.3 A worked trace

Consider the program fragment:

```
let a = 5
let b = 7
print a * b
```

Step	Instruction	`m_stack_size`	Stack (top at right)
1	`mov rax, 5` / `push rax`	1	`[a=5]`
2	`mov rax, 7` / `push rax`	2	`[a=5, b=7]`
3	push `a`: offset = (2−0−1)×8 = 8	3	`[5, 7, 5]`
4	push `b`: offset = (3−1−1)×8 = 8	4	`[5, 7, 5, 7]`
5	`pop rbx` / `pop rax`	2	`[5, 7]`
6	`imul rax, rbx` / `push rax`	3	`[5, 7, 35]`
7	`pop rax` (for printing)	2	`[5, 7]`

At step 3 the variable `a` sits one slot below the top, so its offset is 8 bytes. At step 4, after `a`'s copy has been pushed, `b` is again one slot below the top, so its offset is again 8 bytes, even though the arithmetic differs from step 3.

12.4 Stack balance

The generator is designed so that every expression has a net effect of +1 slot and every non-declaring statement has a net effect of 0. Blocks are balanced by the explicit `add rsp, ...` emitted at their ends. The `time utc` built-in temporarily moves `rsp` by 16 bytes without telling the counter, which is safe only because it restores `rsp` before any other code runs.

12.5 Stack alignment

System V requires `rsp` to be 16-byte aligned at a `call`. Generated programs contain no `call` instructions and no external code, so alignment is irrelevant and is not maintained.

12.6 Program entry

The program starts at `_start`, not `main`. At entry, Linux provides `argc` and the `argv` pointers on the stack, but the generated code ignores them; the language has no access to command-line arguments or environment variables.

---

13. Code Generator Internals

This chapter is aimed at contributors. It explains how `Generator.h` is organised and how to extend it.

13.1 Class structure

Member	Kind	Role
`Generator(NodeProgram root)`	Constructor	Takes ownership of the syntax tree
`gen_expr(const NodeExpression&)`	Public method	Emits code for one expression; leaves one value on the stack
`gen_stmt(const NodeStatements&)`	Public method	Emits code for one statement
`generate()`	Public method	Emits prologue, all statements, epilogue; returns the assembly text
`push(reg)` / `pop(reg)`	Private helpers	Emit `push` / `pop` and update `m_stack_size`
`gen_array_addr(name)`	Private helper	Bounds check and address computation for array access
`m_output`	`std::stringstream`	Accumulates the assembly
`m_vars`	`unordered_map<string, size_t>`	Name → stack slot index
`m_arrays`	`unordered_map<string,`	Array name → element count

Member	Kind	Role
	`size_t>`	
`m_stack_size`	`size_t`	Number of 8-byte slots currently on the stack
`m_*_count`	`size_t`	Per-construct label counters

13.2 Dispatch

Both `gen_expr` and `gen_stmt` select a branch by testing `std::holds_alternative<T>` on a `std::variant` stored in the node (`expr.var` for expressions, `stmt.expr` for statements) and extracting the alternative with `std::get<T>`. Nodes for `if` and `for` are held through `std::shared_ptr`, because they contain statement lists that include further statements, which would otherwise make the type recursive.

If an expression node of an unrecognised type reaches `gen_expr`, no branch matches and *nothing is emitted*. This silently breaks the stack discipline, so every new node type must be given a branch in the same change that introduces it.

13.3 Label naming scheme

Unique labels are formed from a construct prefix, a role word and a decimal identifier taken from a dedicated counter.

Construct	Labels	Counter
`abs`	`abs_done_N`	`m_abs_count`
`max`	`max_done_N`	`m_max_count`
`min`	`min_done_N`	`m_min_count`
`fact`	`fact_loop_N`, `fact_done_N`	`m_fact_count`
`sqrt`	`sqrt_loop_N`, `sqrt_low_N`, `sqrt_done_N`, `sqrt_zero_N`, `sqrt_end_N`	`m_sqrt_count`
`gcd`	`gcd_loop_N`, `gcd_done_N`	`m_gcd_count`
`prime`	`prime_loop_N`, `prime_true_N`, `prime_false_N`, `prime_done_N`	`m_prime_count`
`nth fibonacci`	`fib_loop_N`, `fib_negative_N`, `fib_done_N`	`m_n_fib_count`
`random`	`random_retry_N`, `random_invalid_N`, `random_done_N`	`m_random_count`
`palindrome`	`palindrome_loop_N`, `palindrome_check_N`, `palindrome_false_N`, `palindrome_done_N`	`m_palindrome_count`

Construct	Labels	Counter
`digit sum`	`digitsum_loop_N`, `digitsum_done_N`	`m_digitsum_count`
`power`	`power_loop_N`, `power_done_N`	`m_power_count`
`input`	`input_loop_N`, `input_done_N`, `input_empty_N`, `input_finish_N`	`m_input_count`
`if`	`if_else_N`, `if_end_N`	`m_if_count`
`for`	`for_start_N`, `for_end_N`	`m_for_count`
string literal	`str_N`	`m_string_count`
`count digits`	`count_digits_loop` (fixed)	none
`time utc`	`time_utc_error`, `time_utc_done` (fixed)	none
`print` (integer)	`.print_loop` (local, fixed)	none
bounds failure	`array_oob` (single, global)	none

The last four rows are the inconsistent cases; see Chapter 15.

13.4 Register conventions

The generator uses no persistent register allocation. The following roles are conventional.

Register	Typical role
`rax`	Left operand, accumulator, result of the construct
`rbx`	Right operand; address of an array element
`rcx`	Loop counter, array index, current character in `input`
`rdx`	Remainder from `div` / `idiv`; scratch
`rsi`, `rdi`	Syscall arguments; `rsi` also used as scratch in `sqrt`
`r8` to `r12`	Extra scratch values inside longer built-ins
`rsp`	Stack pointer; base for all variable addressing

Because every operand is parked on the stack between sub-expressions, there is no need to preserve registers across `gen_expr` calls. The only requirement is that a construct finishes with its result pushed.

13.5 Scope snapshots

Blocks (`if` bodies, `else` bodies, `for` bodies) share one mechanism:

1. Copy `m_stack_size`, `m_vars` and `m_arrays` into local constants.
2. Generate the statements of the block.
3. If `m_stack_size` has grown, emit `add rsp, 8 * growth`.

4. Restore the three values from the copies.

Copying the maps makes each block entry O(number of visible names). This is a compile-time cost only and is negligible for typical programs.

13.6 Adding a new built-in: checklist

1. Define the node type in `Parser.h` and add it to the `std::variant` in `NodeExpression`.
2. Teach the tokenizer and parser to recognise the keyword and build the node.
3. Add a `m_<name>_count` counter to the generator's private section.
4. Add an `else if (std::holds_alternative<NodeNew>(expr.var))` branch to `gen_expr`.
5. In that branch: generate each operand, pop them into registers, take a fresh identifier from the counter, emit the instructions with **every label suffixed by the identifier**, and finish with exactly one `push`.
6. Document the semantics, edge cases and overflow behaviour in Chapter 8 of this document.
7. Add test programs covering zero, negative, boundary and nested uses, and a program that uses the construct twice to confirm that labels are unique.

---

14. Error Handling and Diagnostics

14.1 Compile-time diagnostics

All diagnostics produced by the generator are written to standard error and terminate the compiler with `EXIT_FAILURE`. There is no error recovery; the first error ends compilation.

Message	Cause
`Error: undeclared identifier: <name>`	A scalar name was used before declaration or outside its scope
`Error: undeclared array: <name>`	`name[index]`, `len(name)` or an element assignment named an array that does not exist
`Error: identifier already used: <name>`	`let`, array declaration or `for` reused a visible name

Syntax errors are reported by the tokenizer and parser, which are outside the scope of this document.

14.2 Assembler errors

Because the generator performs no check on label uniqueness for the fixed-label constructs, NASM may report errors such as `symbol 'count_digits_loop' redefined` or `symbol '.print_loop' redefined`. These indicate generator limitations (L-01 to L-03), not mistakes in the source program.

14.3 Run-time failures

Condition	Observable result
Array index out of range	Exit status 1, no message

Condition	Observable result
Division by zero, or `INT64_MIN / -1`	Process killed by `SIGFPE` (shell reports status 136)
Stack exhausted by very large arrays	`SIGSEGV` (status 139)
`random` on a CPU without `RDRAND`	`SIGILL` (status 132)
`power` with a negative exponent	Effectively an infinite loop
Normal completion without `exit`	Status 0
`exit n`	Status `n mod 256`

14.4 Debugging suggestions

- Keep the generated `.asm` file and read the section for the failing statement; labels carry the construct name and identifier.
- Run the executable under `gdb` and set a breakpoint at `_start`. Because every variable is addressed from `rsp`, `x/8gx $rsp` shows the live variables.
- Use `strace ./out` to see each `read`, `write` and `exit` syscall with its arguments; this immediately exposes a wrong `write` length (L-05, L-07).
- Test boundaries explicitly: `0`, `1`, `-1`, a large positive number and the empty input.

---

15. Known Limitations and Recommendations

The table below records every limitation identified during review of the code generator, with a severity rating and a recommended remedy. *High* means a valid program can fail to assemble or can produce wrong output in ordinary use. *Medium* means incorrect behaviour at documented boundaries. *Low* means a cosmetic or maintainability issue.

ID	Area	Description	Severity	Recommended remedy
L-01	`count digits`	Labels `count_digits_loop` have no unique suffix; a second use fails to assemble	High	Add an `m_count_digits_count` counter and suffix the label
L-02	`time utc`	Labels `time_utc_error` and `time_utc_done` are fixed; a second use fails to assemble	High	Add a counter and suffix both labels
L-03	`print` (integer)	Local label `.print_loop` is repeated for each print; consecutive prints redefine it	High	Emit a unique global label (for example `print_loop_N`) using the unused `m_print_count`

ID	Area	Description	Severity	Recommended remedy
L-04	`power`	A negative exponent never reaches zero and the loop does not terminate	Medium	Return `0` or `1` for negative exponents, or reject them
L-05	`print` (integer)	Conversion uses unsigned division, so negatives print as large unsigned values; the length passed to `write` is `32 - rcx`, which exceeds digits plus newline	High	Handle the sign explicitly; compute the length as `2 - rcx` (digits plus newline) given the buffer layout
L-06	`sqrt`	`mid × mid` can overflow for arguments above roughly 4×10^9	Medium	Cap `high` at 3 037 000 500 or use an unsigned compare
L-07	`print` (string)	Length is taken from the source text; escape sequences make it too long	High	Compute the assembled length (for example with `$ - str_N` in the data section)
L-08	`random`	Rejection threshold is computed from the lower bound instead of `2^64`; full-range request divides by zero	Medium	Compute `2^64 mod range` from `0:-1` or negate the range; special-case a full range
L-09	`input`	One `read` per call; piped multi-line input is partly discarded	Medium	Buffer input across calls, or read byte by byte until a newline
L-10	Division	No zero-divisor or overflow test	Medium	Test the divisor and exit with a defined status
L-11	Doubles	Stored as bit patterns; no floating-point operations or printing	Medium	Add SSE2 arithmetic and a formatter, or reject double literals
L-12	Scalars	No assignment to a scalar after declaration	Low	Add an assignment statement and node
L-13	Strings	Strings are addresses only; only literal `print` produces text	Low	Add a string type, or print any pointer expression with `strlen`
L-14	`let`	Name is registered before the initialiser is generated, so self-reference is accepted	Low	Register the name after generating the expression

ID	Area	Description	Severity	Recommended remedy
L-15	Overflow	`fact` above 20, `nth fibonacci` above 92, `power` and arithmetic wrap silently	Low	Document, or add `jo` checks that exit with a status
L-16	Arrays	Out-of-bounds access exits with status 1 and no message; arrays share the 8 MB stack	Low	Print a message before exiting; document a maximum size
L-17	Maintainability	`NodeEqual` duplicates `==`; `m_print_count` is declared but unused	Low	Remove the duplicate node or share its code path
L-18	`gcd`	Result can be negative when an argument is negative	Low	Take the absolute value of the result

---

16. Worked Examples

Notation. The examples use illustrative notation. Constructs whose names contain two words are written with an underscore: `nth_fibonacci`, `digit_sum`, `count_digits`, `time_utc`. Expected output is shown after each program. Programs should be adapted to the exact keywords of the tokenizer before use.

16.1 Prime numbers below fifty

Demonstrates: `for`, `if`, the `prime` built-in, integer `print`.

```
for n = 2 to 50 {
    if prime(n) {
        print n
    }
}
```

Output: `2 3 5 7 11 13 17 19 23 29 31 37 41 43 47`, one per line.

16.2 The first ten Fibonacci numbers

Demonstrates: `nth_fibonacci` inside a loop.

```
for i = 0 to 10 {
    print nth_fibonacci(i)
}
```

Output: `0 1 1 2 3 5 8 13 21 34`.

16.3 Statistics over an array

Demonstrates: arrays, `len`, `max`, `min`, one-element arrays as mutable accumulators.

```
let data = [4, 8, 15, 16, 23, 42]
let total = [0]
let hi = [data[0]]
```

```
let lo = [data[0]]
for i = 0 to len(data) {
    total[0] = total[0] + data[i]
    hi[0] = max(hi[0], data[i])
    lo[0] = min(lo[0], data[i])
}
print total[0]
print hi[0]
print lo[0]
```

Output: `108`, `42`, `4`.

16.4 Decimal palindromes between 100 and 199

Demonstrates: `palindrome` as a loop filter.

```
for n = 100 to 200 {
    if palindrome(n) {
        print n
    }
}
```

Output: `101 111 121 131 141 151 161 171 181 191`.

16.5 Least common multiple

Demonstrates: composing `gcd` with arithmetic, `input`.

```
let a = input
let b = input
print a / gcd(a, b) * b
```

With inputs `12` and `18` the output is `36`. Dividing before multiplying reduces the risk of overflow.

16.6 Digit sum of a factorial

Demonstrates: nesting of built-ins.

```
print digit_sum(fact(10))
```

`10! = 3 628 800`, so the output is `27`.

16.7 Classifying a number

Demonstrates: nested conditionals standing in for `else if`.

```
let n = input
if prime(n) {
    print "prime\n"
} else {
    if even(n) {
        print "even composite\n"
    } else {
        print "odd composite\n"
    }
}
```

For input `9` the output is `odd composite`.

16.8 Countdown using index arithmetic

Demonstrates: working around the upward-only `for`.

```
let n = 5
for i = 0 to n {
    print n - i
}
```

Output: `5 4 3 2 1`.

16.9 Exit status as a result

Demonstrates: `exit` with a computed value.

```
let x = 6
let y = x * 7
exit y
```

The process exits with status `42`.

Appendix A. Syntax-Tree Node Reference

Node	Category	Construct
`NodeIntLiteral`	Expression	Integer literal
`NodeDoubleLiteral`	Expression	Double literal (bit pattern)
`NodeStringLiteral`	Expression	String literal
`NodeIntIdentifier`	Expression	Scalar variable reference
`NodeArrayIndex`	Expression	`a[i]`
`NodeLen`	Expression	Array length
`NodeBinary`	Expression	Arithmetic and comparison operators
`NodeEqual`	Expression	Equality
`NodeSum`, `NodeEven`, `NodeAbs`, `NodeMidVal`, `NodeMax`, `NodeMin`	Expression	Numeric built-ins
`NodeFact`, `NodeSqrt`, `NodeGCD`, `NodePrime`, `NodeNthFibonacci`, `NodePower`	Expression	Mathematical built-ins
`NodeDigitSum`, `NodeCountDigits`, `NodePalindrome`	Expression	Digit built-ins
`NodeRandom`, `NodeTimeUTC`, `NodeInput`	Expression	Environment built-ins
`NodeStatementExit`	Statement	`exit`

Node	Category	Construct
`NodeStatementPrint`	Statement	`print`
`NodeStatementLet`	Statement	Scalar declaration
`NodeStatementArrayLet`	Statement	Array declaration
`NodeStatementArrayAssign`	Statement	Element assignment
`NodeIf` (via `shared_ptr`)	Statement	Conditional
`NodeFor` (via `shared_ptr`)	Statement	Counted loop

Appendix B. Operator Quick Reference

Operator	Operation	Result range
`+`, `-`, `*`	Wrapping integer arithmetic	Any 64-bit value
`/`	Truncating signed division	Any 64-bit value; faults on zero divisor
`==`, `!=`, `<`, `>`, `<=`, `>=`	Signed comparison	`0` or `1`

Appendix C. Sample Generated Assembly

The source program

```
let x = 6
let y = x * 7
exit y
```

produces the following assembly (the prologue sections are shown in full).

```
section .bss
input_buffer: resb 64
print_buffer: resb 32
section .text
global _start
_start:
    mov rax, 6              ; let x = 6
    push rax                ;   slot 0
    push QWORD [rsp + 0]    ; x
    mov rax, 7
    push rax                ; 7
    pop rbx
    pop rax
    imul rax, rbx
    push rax                ; let y: slot 1
    push QWORD [rsp + 0]    ; exit y
    mov rax, 60
    pop rdi
    syscall
    mov rax, 60             ; default epilogue
```

```
    mov rdi, 0
    syscall
array_oob:
    mov rax, 60
    mov rdi, 1
    syscall
```

The offsets of both variable reads are zero because, at each read, the variable being copied is the top slot of the stack.

Appendix D. Glossary

Term	Meaning
Slot	One 8-byte position on the machine stack
Stack discipline	The rule that each expression leaves exactly one new value on the stack
Built-in	An expression-level function implemented directly in generated assembly
Syscall	A request to the Linux kernel, issued with the `syscall` instruction
NASM	The Netwide Assembler, which translates the generated text to machine code
`RDRAND`	An x86 instruction returning a hardware-generated random number
Rejection sampling	A technique that discards out-of-range random samples to avoid modulo bias
Two's complement	The signed integer representation used by the processor
Label	A named position in the assembly, used as a jump target

Appendix E. Suggested Roadmap

Priority	Item	Rationale
1	Fix L-01, L-02, L-03, L-05, L-07	These make ordinary programs fail to assemble or print wrong text
2	Add a division-by-zero guard (L-10)	Turns a crash into a defined error
3	Make `input` robust for piped input (L-09)	Required for automated testing
4	Add scalar assignment and a `while` loop	Removes the one-element-array idiom
5	Make `random` exact (L-08) and constrain `power` (L-04)	Correctness at documented boundaries

Priority	Item	Rationale
6	Optimise `prime` (skip even divisors) and `power` (square-and-multiply)	Performance
7	Introduce a test suite of programs with expected output and exit status	Prevents regressions
8	Floating-point support (L-11)	Gives double literals a purpose

---

End of document.